

Task 5:

Debugging and Memory Investigation

1. GDB Debugging Evidence During the development of TaskForge, GDB was utilized iteratively to inspect state transitions and dynamic object structures. Specifically, GDB was used to trace execution within the Task state machine.
 - Breakpoint Setup: A breakpoint was set on the Task::start() method using break Task::start.
 - Stepping and Execution: Upon hitting the breakpoint during simulation execution, the step command was used to step into the state delegation logic (state->handleStart()).
 - State Inspection: Variables and object states were inspected using print *state and print name, confirming that PendingState successfully yielded an InProgressState object and properly cleaned up the previous state pointer to prevent memory leaks.
2. Genuine Bug Encountered During Development
 - Symptom: When executing Scenario 1 (Change Order structural modification), the project root cost remained static at R1000 even after a new "Emergency Repair Bay" task was added mid-execution. Furthermore, initial remote autograder runs failed compilation on Linux with No such file or directory errors for core components.
 - Cause:
 1. A case-sensitivity mismatch between Windows (local WSL environment) and Linux (autograder environment), where code files included #include "WorkGroup.h" while the physical filename on disk was Workgroup.h (lowercase 'g').
 2. The composite container's add() function lacked the underlying children.push_back(child) instruction, meaning newly added change-order tasks were never actually registered in the child vector.
 - Debugging Evidence: Local scenario outputs showed static cost reporting despite structural modifications, and Linux test runners threw fatal object linking failures.
 - Correction:
 1. Standardized all include statements to match the exact lowercase filename (#include "Workgroup.h").

2. Updated the WorkGroup::add() implementation to explicitly push elements into the vector (children.push_back(child)), ensuring recursive cost methods dynamically calculate live totals.
3. Valgrind Memory Investigation Evidence For the final application build, Valgrind was executed using the standard command to check for leaks, invalid reads, or uninitialized memory:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./taskforge
```

Valgrind Output Summary: 2422 Memcheck, a memory error detector 2422 Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al. 2422 Using Valgrind-3.22.0 and LibVEX... 2422 Testing full TaskForge scenarios... --- Executing Scenario 1: Change Order (Structural Modification) --- Original project root cost before change: 1000 Added 'Emergency Repair Bay' mid-execution. Updated project root cost: 1500 --- Scenario 1 Completed Successfully --- ... Scenario Complete Scenarios executed successfully. 2422 2422 HEAP SUMMARY: 2422 in use at exit: 0 bytes in 0 blocks 2422 total heap usage: 34 allocs, 34 frees, 75,568 bytes allocated 2422 2422 All heap blocks were freed -- no leaks are possible 2422 2422 ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Conclusion: The Valgrind report confirms 0 definite memory leaks, 0 errors, and a perfect 1:1 allocation-to-free ratio (34 allocs, 34 frees) across the entire scenario execution lifecycle.